

Parcial de Python

Condicionales · Ciclos · Funciones — Orientado a Ciencia de Datos

🕒 90 min
📄 20 preguntas
🎯 10 puntos

Instrucciones generales

- ▶ Modalidad individual, en computador
- ▶ Solo Python estándar (sin numpy/pandas)
- ▶ Guarde cada respuesta en un archivo .py
- ▶ Sin conexión a internet
- ▶ Cada pregunta vale **0.5 pts**
- ▶ Se evalúa lógica, sintaxis y casos borde

I CONDICIONALES · PREGUNTAS 1 – 5

- 1 Cree `clasificar_imc(peso, altura)` que retorne "Bajo peso", "Normal", "Sobrepeso" u "Obesidad" según el IMC.
- 2 Con `valores = [12,-3,0,45,-7,8]`, imprima si hay negativos usando un condicional (sin `any()`).
- 3 Cree `es_outlier(x, media, desv)` que retorne `True` si `|x-media| > 2*desv`.
- 4 Recorra `ventas = {"enero":100,"febrero":0,"marzo":250}` e indique qué meses no tuvieron ventas.
- 5 Cree `tipo_dato(valor)` que retorne "numérico", "texto", "booleano" u "otro".

II CICLOS FOR · PREGUNTAS 6 – 10

- 6 Calcule el promedio de `edades = [23,45,12,67,34,19]` con un `for`.
- 7 Genere una lista con el cuadrado de `[2,4,6,8,10]` usando `for`.
- 8 Encuentre la temperatura máxima de `temps = [22.5,23.1,21.8,24.0,25.3]` sin usar `max()`.
- 9 Cunte las vocales de "ciencia de datos" con un `for`.
- 10 Con `matriz = [[1,2,3],[4,5,6],[7,8,9]]`, use `for` anidados para imprimir la suma de cada fila.

III CICLOS WHILE · PREGUNTAS 11 – 15

- 11 Imprima del 1 al 10 con un `while`.
- 12 Sume los primeros 100 naturales con `while`.
- 13 Conjetura de Collatz desde `n=27`: cuente iteraciones hasta llegar a 1.
- 14 Simule energía inicial 100 que baja entre 5 y 15 (aleatorio) hasta ≤ 0 . Cuente pasos.
- 15 Recorra `[3,5,7,0,2,8]` con `while` e imprima hasta encontrar un 0 (sin imprimirlo).

IV FUNCIONES DE USUARIO · PREGUNTAS 16 – 20

- 16 `media(lista)`: promedio; retorne `None` si está vacía.
- 17 `normalizar(lista)`: retorne $(x-\min)/(\max-\min)$. Maneje `max==min`.
- 18 `contar_frecuencias(lista)`: retorne diccionario de frecuencias.
- 19 `filtrar_por_umbral(lista, umbral)`: retorne solo valores $>$ umbral.
- 20 `resumen_estadistico(lista)`: retorne `{"min","max","media","n"}`.

Rúbrica y Soluciones

Documento para el docente

 Uso interno



RÚBRICA DE EVALUACIÓN POR PREGUNTA

CRITERIO	PUNTOS
Sintaxis correcta y uso adecuado de estructuras	0.2
Lógica correcta del algoritmo	0.2
Manejo de casos borde (listas vacías, división por cero, etc.)	0.1
Total por pregunta	0.5



SOLUCIONES BREVES (REFERENCIA)

```

# 1
def clasificar_imc(peso, altura):
    imc = peso / altura**2
    if imc < 18.5: return "Bajo peso"
    elif imc < 25: return "Normal"
    elif imc < 30: return "Sobrepeso"
    else: return "Obesidad"

# 3
def es_outlier(x, media, desv):
    return abs(x - media) > 2 * desv

# 5
def tipo_dato(valor):
    if isinstance(valor, bool): return "booleano"
    if isinstance(valor, (int, float)): return "numérico"
    if isinstance(valor, str): return "texto"
    return "otro"

# 6
edades = [23,45,12,67,34,19]
s = 0
for e in edades: s += e
print(s / len(edades))

# 8
temps = [22.5,23.1,21.8,24.0,25.3]
mx = temps[0]
for t in temps:
    if t > mx: mx = t
print(mx)

# 10
matriz = [[1,2,3],[4,5,6],[7,8,9]]
for fila in matriz:
    s = 0
    for x in fila: s += x
    print(s)

# 11
i = 1
while i <= 10:
    print(i); i += 1

# 13
n, pasos = 27, 0
while n != 1:
    n = n // 2 if n % 2 == 0 else 3 * n + 1
    pasos += 1
print(pasos)

# 15
datos = [3,5,7,0,2,8]
i = 0
while i < len(datos) and datos[i] != 0:
    print(datos[i]); i += 1

# 16
def media(lista):
    if not lista: return None
    return sum(lista) / len(lista)

# 17
def normalizar(lista):
    if not lista: return []
    mn, mx = min(lista), max(lista)
    if mx == mn: return [0.0] * len(lista)
    return [(x - mn) / (mx - mn) for x in lista]

# 18
def contar_frecuencias(lista):
    f = {}
    for x in lista:
        f[x] = f.get(x, 0) + 1
    return f

# 19
def filtrar_por_umbral(lista, umbral):
    return [x for x in lista if x > umbral]

# 20
def resumen_estadistico(lista):
    if not lista:
        return {"min":None,"max":None,"media":None,"n":0}
    return {
        "min": min(lista),
        "max": max(lista),
        "media": sum(lista) / len(lista),
        "n": len(lista)
    }

```